Light Heterogeneous Hypergraph Contrastive Learning Based Service Recommendation for Mashup Creation

Mingdong Tang, *Member, IEEE*, Jiajin Mai, Fenfang Xie, and Zibin Zheng, *Fellow, IEEE*

***Abstract*—Mashup technology enables developers to create new applications more readily by combining existing services. As its popularity grows, research on service recommendation for mashup creation has gained increasing attention. Existing recommendation methods have the following limitations: either they are susceptible to data sparsity problems, or they exhibit over-smoothing when aggregating high-order neighbors, resulting in similar and non-specific node feature representations, or they only focus on bipartite graphs and neglect the rich heterogeneous information in the mashup-service ecosystem. To address these issues, we propose a service recommendation method for mashup creation based on light heterogeneous hypergraph contrastive learning (LHGCL). This method first constructs a heterogeneous hypergraph by combining mashup information, service information, the mashup-service interaction data, and their related attribute information. Then, it designs a light hypergraph neural network to capture the high-order relationships between mashups and services. Next, it applies contrastive learning to enhance the representations of mashups and services. Finally, it utilizes the enhanced feature vectors of mashups and services to predict mashup preferences for services. Comprehensive experiments conducted on the real-world ProgrammableWeb dataset demonstrate the superiority of the proposed method and the effectiveness of its key modules.**

***Index Terms*—Service recommendation, heterogeneous hypergraph, hypergraph neural network, contrastive learning.**

I. Introduction

WITH the wide spread of Web 2.0, a new technology, designated "mashup", has emerged on the Internet, which is a creative way to develop a new application by reusing the existing services [1]. A well-known example is the composition of *Google Maps* and *Weather Underground* services into a new mashup application that users can check the local weather while using map navigation, providing a more convenient experience for users. When mashup developers encounter development requirements, they can browse a large number of services offering distinct functionalities on the public mashup platform. From there, they can select suitable services to compose, resulting in mashup applications endowed with enhanced functionalities. However, with the rapid increase in the number of services on the Internet, it is difficult for inexperienced mashup developers to choose suitable services by browsing popular services or searching manually. Therefore, the intelligent service recommendation system comes into being, which plays a crucial role in providing a personalized recommendation list for each mashup developer so that they can find suitable services for mashup creation conveniently.

To address the challenge of service information overload, researchers have developed various methods for service recommendation. One prominent technique is collaborative filtering (CF), widely utilized for mining preferences from past mashup-service interactions. Traditional CF methods assume that similar mashups tend to share similar service preferences [2], [3], [4], [5], [6]. Recently, graph neural networks (GNNs) have emerged as powerful tools in recommendation systems [7], [8], [9], [10], [11], [12]. GNNs extend neural network models to process graph-structured data, effectively capturing relationships and dependencies among entities. By iteratively aggregating and updating features associated with each node based on neighborhood information, GNNs propagate information throughout the graph. Additionally, hypergraphs expand the neighborhood of nodes, allowing for the aggregation of richer higher-order information and more accurate modeling of multivariate associations. Drawing from the ability of hypergraphs to represent complex high-order dependencies, research has explored hypergraph neural networks (HGNNs) for recommendation systems [13], [14], [15], [16], [17], [18]. Furthermore, contrastive learning (CL) has recently gained traction in the recommendation domain [19], [20], [21], [22], [23], owing to its ability to glean additional insights from original data and derive high-quality node representations from limited interaction data. A typical approach in contrastive learning involves augmenting the mashup-service bipartite graph into different views and maximizing the consistency of the node representation across these views [24], [25], [26], [27], [28]. CL serves as a complementary technique in recommendation tasks, further enhancing performance.

Although the aforementioned methods have yielded satisfactory outcomes in terms of service recommendation, they

Received 18 June 2024; revised 17 October 2024; accepted 25 October 2024. Date of publication 31 October 2024; date of current version 30 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61976061, Grant 62102461, and Grant 61572186. *(Corresponding author: Fenfang Xie.)*

Mingdong Tang, Jiajin Mai, and Fenfang Xie are with the School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou 510006, China (e-mail: mdtang@gdufs.edu.cn; 2479758764@qq.com; xiefragrance@gmail.com).

Zibin Zheng is with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China.

Digital Object Identifier 10.1109/TSC.2024.3489417

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

still have some limitations: (1) it is difficult for traditional CF methods to find similar mashups or services when facing data sparsity issues, which causes insufficient recommendation ability. (2) GNN-based methods may suffer from the over-smoothing issue, which may cause indistinguishable node representations and a decline in recommendation quality. In addition, the simple graph expresses the relationship between the nodes in the graph as a binary relationship, which loses many high-order relationships in the original data. (3) Most of the existing HGNN-based methods focus on homogeneous hypergraphs, and only a few consider bipartite graphs, and do not consider the rich heterogeneous information in the mashup-service ecosystem.

To tackle the above limitations and further improve the accuracy of service recommendations, this paper integrates hypergraph neural network and contrastive learning techniques, and incorporates heterogeneous information to construct a novel model for recommending suitable services for mashup creation. Specifically, it first constructs a heterogeneous hypergraph by combining mashup information, service information, interaction records between mashups and services, and their respective attribute information. Then, it implements a light hypergraph neural network model that leverages the rich heterogeneous information from the perspectives of mashups and services to explore high-order associations between them. Next, it designs two contrastive modules, namely, the heterogeneous mashup hypergraph module and the heterogeneous service hypergraph module, to enhance the feature vectors of mashups and services by maximizing consistency among positive samples and minimizing consistency among negative samples. Finally, it utilizes the enhanced feature vectors of mashups and services to predict mashup preferences for services. Experimental results demonstrate that the proposed method consistently outperforms the strongest baselines, with improvements ranging from 20.7% to 30.4% in Recall and 35.4% to 38.7% in NDCG. Further, the proposed light hypergraph neural network framework, the attribute information, and the contrastive learning all have positive effects on improving the recommendation performance.

The major contributions of this paper are summarized as follows:

- We propose a light hypergraph neural network framework for aggregating high-order neighbor information of mashups and services. This framework not only simplifies model design but also helps improve the recommendation performance.
- We integrate their respective attribute information from the perspective of mashup and service, and introduce contrastive learning to enhance the modules from these two perspectives. In this way, we can refine the feature vector representation of mashups and services.
- We conduct extensive experiments on a real-world dataset crawled from ProgrammableWeb.com, and the experimental results show the superiority of the proposed model, as well as the rationality and effectiveness of the designed modules.

II. Related Work

Numerous technologies have been utilized in the field of service recommendation. Our approach focuses on utilizing graph neural networks, hypergraph neural networks, and contrastive learning techniques. Consequently, we mainly summarize the research on recommendation systems related to these technologies. We broadly categorize recommendation methods into three types: GNN-based, HGNN-based and CL-based methods.

A. GNN-Based Methods

Graph Neural Networks are deep learning models designed for processing graph-structured data. GNNs typically consist of multiple layers, each processing information from neighboring nodes to update the node representations iteratively [29]. This enables them to capture complex dependencies and patterns within the graph data. GNN based methods have achieved greater than ever performances in different scenarios. For example, Ying et al. [30] propose a model named Pinsage which designs an efficient sampling method based on a random walk to handle large-scale datasets. NGCF [7] largely follows the standard GCN [31] which encodes the high-order information between users and items. LightGCN [32] removes redundant operations in GCN including transformation matrices and nonlinear activations during message passing. Wang et al. [33] propose a model called DGCF which disentangles latent user interests to learn a more complicated representation. Jin et al. [12] use a simplified graph convolution operation to increase the training efficiency and remove the heterogeneity of the bipartite graph. Zhang et al. [34] first learn the behavioral and auxiliary information by recursively performing graph neural networks and then design the hierarchical interaction layer to model the importance between behavioral and auxiliary information which further improves recommendation performance.

B. HGNN-Based Methods

Hypergraphs are generalizations of graphs that allow for relationships between more than two entities. Hypergraph [35] provides a natural way to model complex high-order dependencies. Hypergraph neural networks aim to capture the complex dependencies and interactions present in hypergraph-structured data by aggregating and processing information from hyperedges and their associated nodes. By considering higher-order relationships, hypergraph neural networks can effectively model intricate data structures found in various domains, such as knowledge graphs, recommendation systems, and biological networks. HGNN [13] is the first work that designs a hypergraph convolution operation to handle the data correlation during representation learning. Bai et al. [15] propose hypergraph attention to hypergraph convolutional networks to improve performance. HyperRec [36] models the short-term user preference by considering users as hyperedges for next-item recommendation. DHCF [37] proposes a dual-channel learning strategy that can model the high-order correlations among users and items based on the constructed hypergraph structures. Pent et al. [38] propose

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

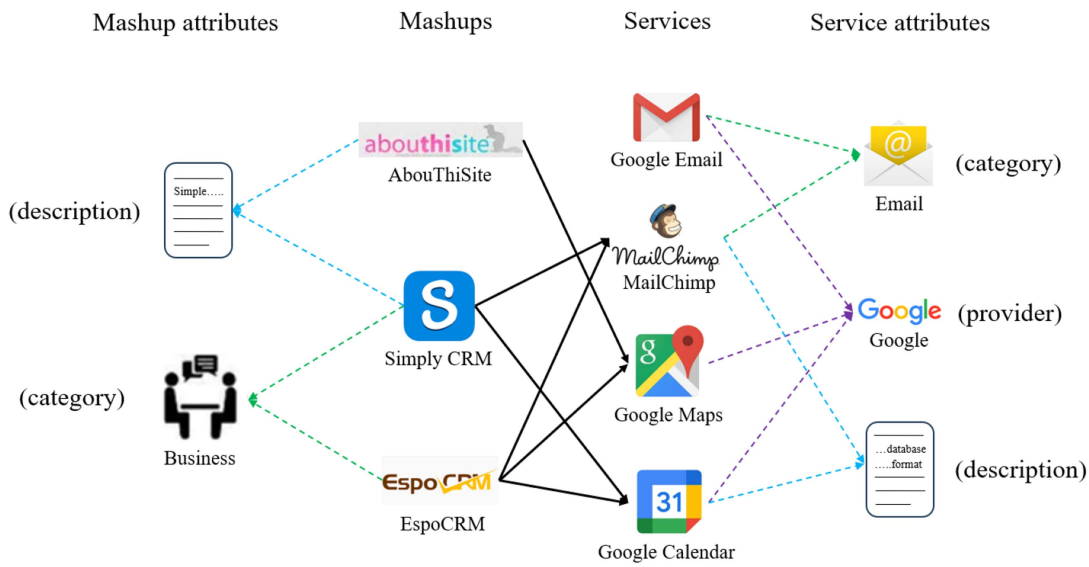


Fig. 1. The Mashup-Service ecosystem.

a new learning scheme to capture important graph features by replacing neighborhood aggregation.

C. CL-Based Methods

Contrastive learning based methods recently have attracted much attention in many fields because of their ability to extract additional information from massive unlabeled data [24]. They learn to map data instance pairs closer together in the embedding space if they are similar and farther apart if they are dissimilar. By doing so, the network aims to capture meaningful and discriminative features that can effectively distinguish between different classes or categories in the data. As contrastive learning usually works in a self-supervised framework, it is inherently a possible solution to the data sparsity issue in recommendation systems. Inspired by the achievements in other fields, there is some new research that integrates CL-based methods in recommendations. The most widely used model is SGL [24] which performs the node/edge dropout to augment the user-item graph, then implements it on the LightGCN model. Lin et al. [39] propose a model called NCL which designs a heuristic-based strategy to capture the correlations between users and items and their context. SimGCL [40] performs data augmentation with random noise perturbation instead of graph augmentations. Cai et al. [25] propose a simple yet effective graph contrastive learning paradigm LightGCL that utilizes singular value decomposition for contrastive augmentation.

However, most of the current GNN-based methods are limited by the over-smoothing issue when aggregating high-order neighbor information, which may lead to indistinguishable representations, resulting in the decline of recommendation quality. Most of the existing HGNN-based methods only focus on homogeneous hypergraphs, and only several studies consider bipartite graphs, which lose rich heterogeneous information. Different from the above approaches, this paper first utilizes heterogeneous hypergraph neural network to mitigate the over-smoothing issue and leverage the attribute information, and then applies contrastive learning techniques to alleviate the issue of data sparsity and attempts to learn meaningful and discriminative feature representations for mashups and services.

III. Preliminaries

This section gives the formal definitions and summarizes the key notations used in this paper.

Definition 1: Mashup-Service Ecosystem. A mashup-service ecosystem is defined as a complex system consisting of mashups and services, and their respective attributes. It is represented as a quadruple $\mathcal{ECO} = \{\mathcal{M}, \mathcal{S}, \mathcal{A}_{\mathcal{M}}, \mathcal{A}_{\mathcal{S}}\}$, where $\mathcal{M}$ is the mashup set, $\mathcal{S}$ is the service set, $\mathcal{A}_{\mathcal{M}}$ is the mashup attributes set, and $\mathcal{A}_{\mathcal{S}}$ is the service attributes set. The number of mashups and services are M and N, and the indexes m and s are used to denote a mashup and a service, respectively.

Fig. 1 shows a toy example of the mashup-service ecosystem, which has a variety of mashup information, service information, their respective attribute information, and different types of relationships among them. For example, Mashup *EspoCRM* is a composite of services *MailChimp*, *Google Maps* and *Google Calendar*. Services *Google Email*, *Google Maps*, and *Google Calendar* are all provided by the same provider *Google*. Mashups *Simply CRM* and *EspoCRM* belong to the same category, and services *MailChimp* and *Google Email* also belong to the same category.

Definition 2: Hypergraph. A hypergraph can be defined by $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{H})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the node set and the hyperedge set, respectively. $\mathcal{H}$ is a $|\mathcal{V}|$ Ã $|\mathcal{E}|$ incidence matrix that describes the connectivity of the hypergraph. An entry of $\mathcal{H}$ can be formally defined as follows:

$$h(v, e) = \begin{cases} 1, & v \in e \\ 0, & v \notin e \end{cases} \tag{1}$$

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Descriptions
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{H})$	A hypergraph
$\mathcal{V}$	The set of nodes
$\mathcal{E}$	The set of hyperedges
$\mathcal{H}$	The incidence matrix
$\mathcal{A}$	The predefined sets of node types
$\mathcal{R}$	The predefined sets of hyperedge types
$E_V^{(0)}$	The embedding table that stores the initial representations
$e_m^{(l+1)}/e_s^{(l+1)}$	The representation of mashup/service in the $(l+1)$ layer
D_m/D_s	The diagonal matrix of mashup/service degree
$D_{\mathcal{E}_m}/D_{\mathcal{E}_s}$	The diagonal matrix of mashup/service hyperedge degree
W_m/W_s	The diagonal matrix of mashup/service that stores all the positive weight of hyperedges
θ, δ	The learnable weight matrices
e_m/e_s	The final representation of mashup/service

where $v \in \mathcal{V}$ and $e \in \mathcal{E}$. When $v \in$ e, that v is connected by e, $h(v, e) = 1$, otherwise $h(v, e) = 0$.

The primary distinction between a hypergraph and a traditional graph lies in their representation of relationships between entities. In a standard graph, an edge represents the relationship between two entities, whereas in a hypergraph, a hyperedge represents the relationship among multiple entities. This enables a more intricate and refined description of relationships within certain scenarios.

Definition 3: Heterogeneous Hypergraph. A heterogeneous hypergraph can be also defined by $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{H})$, and it has a node type mapping function ϕ: $\mathcal{V} \rightarrow \mathcal{A}$ and a hyperedge type mapping function ψ: $\mathcal{E} \rightarrow \mathcal{R}$. $\mathcal{A} = \{p_1, ...\}$ and $\mathcal{R} = \{r_1, ...\}$ denote the predefined sets of node types and hyperedge types, respectively, with $|\mathcal{A}| + |\mathcal{R}| > 2$.

In our heterogeneous hypergraph, we define two types of nodes: mashups and services. For each type of node, different types of hyperedges are defined.

For mashups, we define three types of hyperedges:

- Co-service: the connected mashups invoke the same service.
- Co-category: the connected mashups belong to the same category.
- Co-description: the connected mashups have similar functional descriptions.

For services, we define four types of hyperedges:

- Co-mashup: the connected services are invoked by the same mashup.
- Co-category: the connected services belong to the same category.
- Co-provider: the connected services are with the same provider.
- Co-description: the connected services have similar functional descriptions.

Table I depicts the notations used in this paper.

IV. METHODOLOGY

This section introduces the overall framework of the proposed method, followed by a detailed description of its three main components.

A. The Overall Framework

The overall framework of the proposed method is shown in Fig. 2. It mainly consists of three components:

- *Heterogeneous Hypergraph Constructing:* It constructs heterogeneous hypergraphs by combining mashup information, service information, mashup-service interaction data, and their respective attribute information. After that, we get the heterogeneous mashup hypergraph and the heterogeneous service hypergraph.
- *Light Heterogeneous Hypergraph Neural Network:* It designs a light hypergraph neural network by removing transfer matrices and nonlinear activation operations during message passing. The component iteratively aggregates the higher-order association information of nodes in heterogeneous mashup hypergraphs and heterogeneous service hypergraphs to capture the global structural information of the corresponding hypergraphs. As a result, it learns complex higher-order dependencies and purifies the feature representations of mashup and service.
- *Contrastive Learning:* It designed two contrastive modules for heterogeneous mashup hypergraph and heterogeneous service hypergraph to enhance the learning of feature vectors for mashups and services by maximizing the consistency between positive samples and minimizing the consistency between negative samples.

Each component will be described in detail in the following subsections.

B. Heterogeneous Hypergraph Constructing

An example of a heterogeneous hypergraph built from our dataset is shown in Fig. 3. Three types of hyperedges are defined for mashups, namely, co-service, co-category and co-description. Four types of hyperedges are defined for services, namely, co-mashup, co-category, co-provider and co-description. Specially, for the attribute information of the description, we adopt the pre-trained model BERT [41] to convert the description into a 512-dimensional representation, and then use cosine similarity to measure the similarity between these descriptions. At last, these descriptions will be divided into different hyperedges by setting a similarity threshold t.

We adopt the stack strategy which concatenates different hyperedges together to obtain the complete hypergraph $\mathcal{H}_m \in \mathbb{R}^{M \times \mathcal{E}_m}$ of mashups, and $\mathcal{H}_s \in \mathbb{R}^{N \times \mathcal{E}_s}$ of services, which can be formulated as,

$$\mathcal{H} = \mathcal{H}^{(0)} || \mathcal{H}^{(1)} || \ldots || \mathcal{H}^{(n)} \tag{2}$$

where $||$ denotes concatenation operation, $\mathcal{H}^{(0)}$ to $\mathcal{H}^{(n)}$ denote incidence matrices constructed by different hyperedges, respectively.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

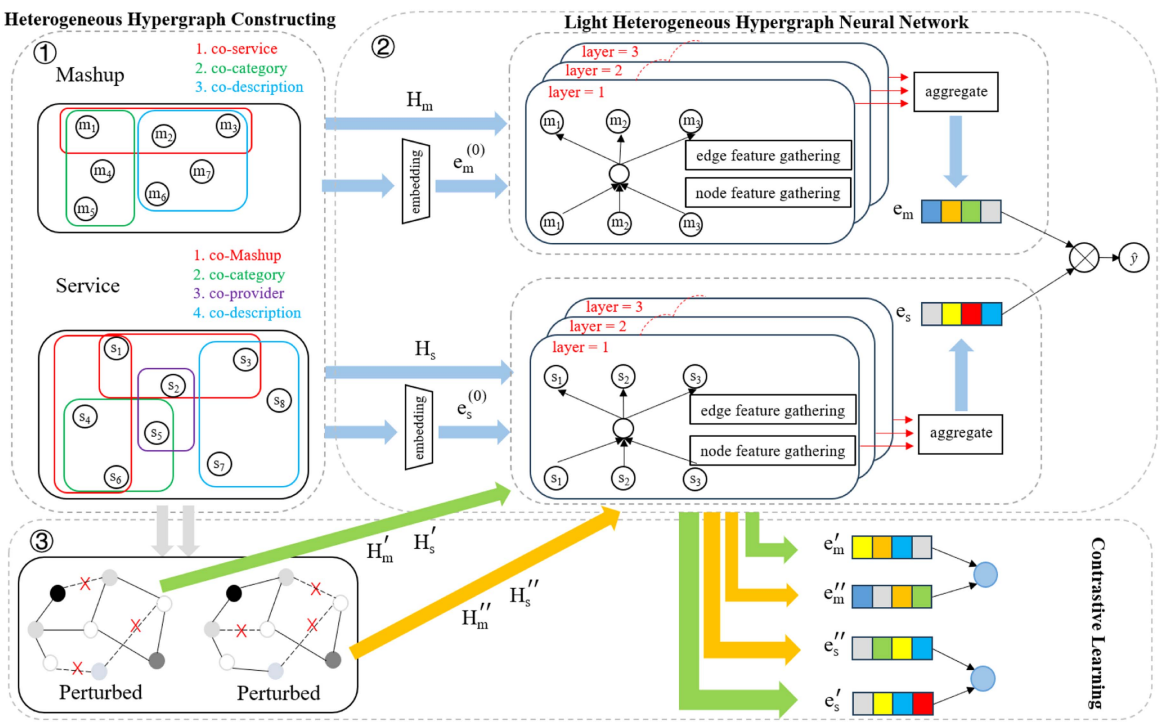


Fig. 2. The overall framework of the proposed approach.

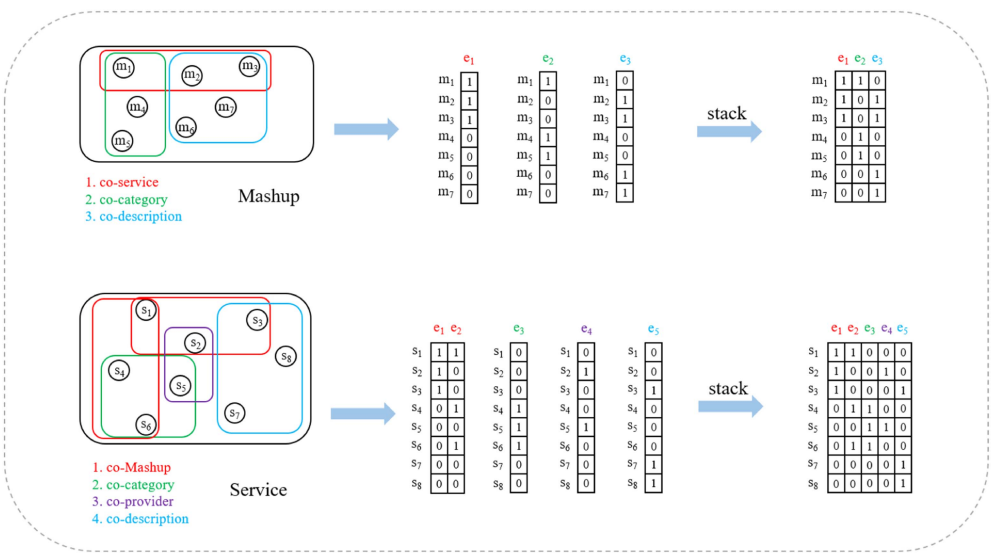


Fig. 3. An example of the heterogeneous hypergraph construction.

C. Light Heterogeneous Hypergraph Neural Network

First, we map each mashup and service into a low-dimensional dense representation via embedding technology:

$$E_V^{(0)} = [e_{m_1}^{(0)}, \ldots, e_{m_M}^{(0)}, e_{s_1}^{(0)}, \ldots, e_{s_N}^{(0)}] \tag{3}$$

where $E_V^{(0)} \in \mathbb{R}^{(M+N)\times d}$ is the embedding table that stores the initial representations of mashups and services, d represents the size of the embedding vector.

After obtaining the heterogeneous hypergraph and the initial representations, the hypergraph neural network [13] is adopted to aggregate high-order neighbor information to update node representations. The message passing scheme can be mainly divided into two stages: the first stage is from nodes to hyperedges, and for each hyperedge, its representation can be obtained by gathering related node representations. The second stage is from hyperedges to nodes, for each node, its representation can be obtained by gathering related hyperedge representations. The original hypergraph convolutional layer is defined in the

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

following formulation:

$$e_m^{(l+1)} = \sigma(D_m^{-1/2}\mathcal{H}_m W_m D_{\mathcal{E}_m}^{-1}\mathcal{H}_m^\top D_m^{-1/2} e_m^{(l)}\theta^{(l)}) \quad (4)$$

$$e_s^{(l+1)} = \sigma(D_s^{-1/2}\mathcal{H}_s W_s D_{\mathcal{E}_s}^{-1}\mathcal{H}_s^\top D_s^{-1/2} e_s^{(l)}\delta^{(l)}) \quad (5)$$

$$d(v) = \sum_{e\in\mathcal{E}_v} W(e)h(v,e) \quad (6)$$

$$d(e) = \sum_{v\in\mathcal{V}} h(v,e) \quad (7)$$

where $e_m^{(l+1)}$ and $e_s^{(l+1)}$ denote the representation of mashup and service in the $(l+1)$ layer, respectively. D_m and $D_{\mathcal{E}_m}$ are the diagonal matrices of the mashup degree and hyperedge degree, respectively. Similarly, D_s and $D_{\mathcal{E}_s}$ are the diagonal matrices of the service degree and hyperedge degree, respectively. W_m and W_s are diagonal matrices that stores all the positive weight of hyperedges, respectively. θ and δ are both learnable weight matrices and $\sigma(\cdot)$ denotes the non-linear activation function. $W(e)$ denotes the positive weight of a hyperedge e, $d(v)$ and $d(e)$ denote the specific degree of a node v and a hyperedge e, respectively.

Inspired by LightGCN [32], we remove the transformation matrices and non-linear activations during message passing to light the burden of the model, which is formally described as follows:

$$e_m^{(l+1)} = D_m^{-1/2}\mathcal{H}_m W_m D_{\mathcal{E}_m}^{-1}\mathcal{H}_m^\top D_m^{-1/2} e_m^{(l)} \quad (8)$$

$$e_s^{(l+1)} = D_s^{-1/2}\mathcal{H}_s W_s D_{\mathcal{E}_s}^{-1}\mathcal{H}_s^\top D_s^{-1/2} e_s^{(l)} \quad (9)$$

After passing the message through L layers, the embedding vectors are fused by using sum pooling to aggregate the neighborhood information in each layer. The final representations of mashup m and service s are e_m and e_s:

$$e_m = \sum_{l=0}^{L} e_m^{(l)} \quad (10)$$

$$e_s = \sum_{l=0}^{L} e_s^{(l)} \quad (11)$$

Finally, the interaction preference score between the mashup and the service is predicted using the inner product. This is formally presented as follows:

$$\hat{y}(m,s) = e_m^\top e_s \quad (12)$$

where $\hat{y}(m,s)$ is the predicted preference score.

To learn model parameters, we optimize the Bayesian Personalized Ranking (BPR) loss for pairwise learning [42], which is a loss function based on sampled triples by maximizing the difference between observed interactions and unobserved ones:

$$\mathcal{L}_{main} = \sum_{(m,s,s')\in O} -ln\sigma(\hat{y}(m,s) - \hat{y}(m,s')) + \lambda_1\|\Theta\|_2^2 \quad (13)$$

where $O = \{(m,s,s') \mid (m,s)\in R^+,\ (m,s')\in R^-\}$ denotes the training data, $\sigma(\cdot)$ is a sigmoid function. R^+ and R^- indicate the observed interactions and unobserved interactions, respectively. Θ is the set of model parameters and λ_1 is a hyperparameter to control the L_2 regularization to prevent overfitting. We treat BPR loss as the main loss function.

D. Contrastive Learning

Contrastive Learning is a type of unsupervised learning algorithm that aims to learn representations by contrasting positive pairs (similar nodes) against negative pairs (dissimilar nodes) [43]. The basic idea behind contrastive learning is to bring similar instances closer in the embedding space while pushing dissimilar instances farther apart.

In our proposed approach, we design the contrastive learning component by maximizing the node representation consistency between different views. In particular, we adopt the edge dropout operator over both heterogeneous hypergraph $\mathcal{H}_m$ and $\mathcal{H}_s$ to generate two views:

$$\begin{cases}\mathcal{H}'_m = M_1 \odot \mathcal{H}_m \\ \mathcal{H}''_m = M_2 \odot \mathcal{H}_m\end{cases} \quad (14)$$

$$\begin{cases}\mathcal{H}'_s = S_1 \odot \mathcal{H}_s \\ \mathcal{H}''_s = S_2 \odot \mathcal{H}_s\end{cases} \quad (15)$$

where M_1, $M_2 \in R^{M\times\mathcal{E}_m}$ and S_1, $S_2 \in R^{N\times\mathcal{E}_s}$ are binary masking matrices. $\odot$ denotes the element-wise multiplication. $\mathcal{H}'_m$, $\mathcal{H}''_m \in R^{M\times\mathcal{E}_m}$ and $\mathcal{H}'_s$, $\mathcal{H}''_s \in R^{N\times\mathcal{E}_s}$ are disturbed views of a mashup and a service, respectively.

Repeating the steps in Section IV-C, that is, light hypergraph convolution and multilayer neighborhood information aggregation, we obtain the augmented representation e'_m, e''_m and e'_s, e''_s.

In our proposed approach, we take different views of the same node as the positive pairs (i.e., e'_m, e''_m), and treat views of different nodes as the negative pairs (i.e., e'_m, $e''_{m'}$). By doing so, our method learns better representations by the supervision of positive pairs and negative pairs. We formally adopt the contrastive loss with the InfoNCE [44], to maximize the agreement of positive pairs and minimize the agreement of negative pairs:

$$\mathcal{L}_c^{mashup} = \sum_{m\in M} -log\frac{\exp(\mathrm{sim}(e'_m, e''_m)/\tau)}{\sum_{m'\in M}\exp(\mathrm{sim}(e'_m, e''_{m'})/\tau)} \quad (16)$$

$$\mathcal{L}_c^{service} = \sum_{s\in N} -log\frac{\exp(\mathrm{sim}(e'_s, e''_s)/\tau)}{\sum_{s'\in N}\exp(\mathrm{sim}(e'_s, e''_{s'})/\tau)} \quad (17)$$

where $\mathrm{sim}(\cdot)$ denotes the cosine similarity and τ denotes the tunable temperature hyperparameter to adjust the scale for softmax [45]. The contrastive loss is jointly optimized with the main loss for the recommendation task:

$$\mathcal{L} = \mathcal{L}_{main} + \lambda_2(\mathcal{L}_c^{mashup} + \mathcal{L}_c^{service}) \quad (18)$$

where λ_2 is a hyperparameter to control the strengths of contrastive loss. We minimize the loss function by using Adam optimizer [46]. Algorithm 1 summarizes the proposed method.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTICS OF THE DATASET

Relation A-B	Number of A	Number of B	Number of A-B
Mashup - Service	2,778	1,207	9,171
Mashup - Category	2,778	352	8,473
Mashup - Description	2,778	2,778	2,778
Service - Category	1,207	128	1190
Service - Provider	1,207	271	384
Service - Description	1,207	1,207	1,207

Algorithm 1: Parameter Learning of LHGCL Model.

Require: Set of initialized mashup/service features, set of training instances O, number of layers L, embedding size d, regularization coefficient λ_1, the magnitude of contrastive learning λ_2.
Ensure: Model parameters Θ
Initialize model parameters Θ randomly
Construct the heterogeneous hypergraph $\mathcal{H}_m$ and $\mathcal{H}_s$
for epoch $= 1, \ldots,$ Epochs **do**
  Generate the disturbed views $\mathcal{H}'_m$, $\mathcal{H}''_m$ and $\mathcal{H}'_s$, $\mathcal{H}''_s$ according to (14), (15), (16) and (17)
  for $l = 1, 2, \ldots, L$ **do**
    Calculate $e_m^{(l)}$, $e_m^{'(l)}$, $e_m^{''(l)}$ and $e_s^{(l)}$, $e_s^{'(l)}$, $e_s^{''(l)}$ according to (8) and (9)
  end for
  Obtain the final representations e_m, e_s and the augmented representations e'_m, e''_m and $e_s{}'$, e''_s according to (10) and (11)
  for $(m, s, s') \in O$ **do**
    Calculate the predicted scores $\hat{y}(m, s)$ and $\hat{y}(m, s')$ according to (12)
    Calculate the loss according to (13), (18), (19) and (20)
    Update model parameters Θ by Adam
  end for
end for
return Model parameters Θ

V. EXPERIMENTS

This section outlines the experimental setup, which includes the dataset, evaluation metrics, baseline models, and implementation details. Next, the effectiveness of the proposed model is evaluated in comparison to other comparative methods. Subsequently, a series of parametric experiments are conducted to investigate parameter settings, while ablation experiments are performed to validate the designed module's soundness.

A. Experimental Setup

Dataset: A real-world dataset is obtained by crawling the well-known mashup management platform ProgrammableWeb website[1]. The dataset comprises 2,778 mashups, 1,207 services, and 9,171 mashup-service interactions. A detailed description of the statistics is presented in Table II. In addition, Tables III and IV provide separate examples of mashups and services, both containing heterogeneous information. It is apparent that both the mashup and service possess numerous related attributes, enriched with semantic information, which can be utilized to enhance the model's capability of learning the representations of both mashup and service. Consequently, this is expected to enhance the recommendation performance.

[1][Online]. Available: https://www.programmableweb.com/

TABLE III
AN EXAMPLE OF A MASHUP WITH ITS RELATED ATTRIBUTES

Name	EspoCRM
Services	Google Maps, Google Calendar, Google Contacts, MailChimp, Twilio
Category	Customer Relationship Management, Business, Customer Service, Marketing, Open Source, Sales
Description	EspoCRM is a free open-source web application that allows you to see, enter and analyze your company's relationships with customers and partners alike. Lightning fast and easily customizable, this web-based CRM solution will provide you with the tools to understand your consumers' behavior and cater your products and services specifically to their needs.

TABLE IV
AN EXAMPLE OF A SERVICE WITH ITS RELATED ATTRIBUTES

Name	Google Maps
Category	Mapping
Provider	Google
Description	The Google Maps API allow for the embedding of Google Maps onto web pages of outside developers, using a simple JavaScript interface or a Flash interface. It is designed to work on both mobile devices as well as traditional desktop browser applications. The API includes language localization for over 50 languages, region localization and geocoding, and has mechanisms for enterprise developers who want to utilize the Google Maps API within an intranet. The API HTTP services can be accessed over a secure (HTTPS) connection by Google Maps API Premier customers.

Evaluation Metrics: To estimate the effectiveness of our LHGCL method and other baselines, we adopt two metrics: Recall@K and NDCG@K, which have been widely used in various recommendation systems [7], [32]. Recall@K refers to the proportion of relevant services retrieved in the top-K recommended services. NDCG@K is a measure of ranking quality in recommendation systems. They can be formally defined as follows:

$$Recall@K = \frac{1}{|M|} \sum_{m \in M} \frac{|R^K(m) \bigcap T(m)|}{|T(m)|} \tag{19}$$

$$NDCG@K = \frac{1}{|M|} \sum_{m \in M} \frac{\sum_{k=1}^{K} \frac{I(R_k^K(m) \in T(m))}{log(k+1)}}{\sum_{k=1}^{K} \frac{1}{log(k+1)}} \tag{20}$$

where $R^K(m)$ denotes a list of top-K recommended services. $T(m)$ represents a list of observed services in the test set. $R_k^K(m)$ denotes the kth item in the recommended list $R^K(m)$.

Baseline Methods: To evaluate the performance of our proposed method, we compare it with MF-based methods

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT METHODS IN RECALL@K

Methods	Recall@5	Recall@10	Recall@15	Recall@20
BPRMF	0.39069	0.47721	0.52439	0.55882
NGCF	0.34837	0.43658	0.48510	0.52370
LightGCN	0.40885	0.49423	0.53716	0.56507
SGL	0.41665	0.49810	0.53829	0.56683
HGNN	0.40947	0.49651	0.54616	0.58290
LHGCL	**0.54319**	**0.62563**	**0.66965**	**0.70373**
Improve.%	**30.4%**	**25.6%**	**22.6%**	**20.7%**

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT METHODS IN NDCG@K

Methods	NDCG@5	NDCG@10	NDCG@15	NDCG@20
BPRMF	0.28953	0.31698	0.32978	0.33800
NGCF	0.25631	0.28676	0.30009	0.30671
LightGCN	0.30700	0.33453	0.34746	0.35493
SGL	0.32132	0.34749	0.35706	0.36435
HGNN	0.30776	0.33542	0.34777	0.35641
LHGCL	**0.44580**	**0.47379**	**0.48531**	**0.49321**
Improve.%	**38.7%**	**36.3%**	**35.9%**	**35.4%**

(BPRMF), GNN-based methods (NGCF and LightGCN), HGNN-based methods (HGNN), and CL-based methods (SGL):

- *BPRMF* [42]: a classic matrix factorization method using the BPR loss function, which optimizes the embedding through pairwise ranking.
- *NGCF* [7]: a GNN based collaborative filtering method that propagates embeddings with high-order connection information.
- *LightGCN* [32]: a simplified GNN based method that removes the transformation matrices and non-linear activations during message passing and uses sum pooling to generate the final representations.
- *HGNN* [13]: the first work that designs a hypergraph convolution operation in representation learning from a spectral perspective.
- *SGL* [24]: a self-supervised GNN based method directly changes the structures of interaction graphs for data augmentation with 1) node and edge dropout operations; 2) random walk-based sampling, then implements on the LightGCN model.

Implementation Details: The dataset is divided into two subsets, a training set and a test set, with a ratio of 80:20, respectively. 10% random samples from the training set are selected as a validation set to tune hyperparameters. To mitigate the sampling bias, the performance is evaluated using the all-ranking protocol [32] over all items. The proposed method is implemented in PyTorch[2]. The default settings for the similarity threshold t, embedding size d, learning rate, layer number L and the two regularization parameters λ_1 and λ_2 are 0.6, 256, 0.001, 4, 0.01, 0.01, respectively. To ensure a fair comparison, all methods are trained from scratch and initialized with the Xavier Initialization [47]. The batch size is set to 2048.

B. Performance Comparison

Tables V and VI show the experimental results of our proposed method and the comparative methods. The following observations are made:

- The proposed method, LHGCL, outperforms all of the baselines in all evaluation metrics. The observed improvements can be attributed to the following factors: 1) The proposed light hypergraph framework alleviates the model's computational burden while effectively capturing high-order neighbour information. 2) LHGCL integrates attribute information to explore deep relationships between mashups and services, thus enhancing representation learning. 3) The utilisation of contrastive learning enables the maintenance of the inherent characteristics and correlations of mashup preferences from different views.
- BPRMF exhibits superior performance to that of NGCF. This is attributed to the possibility that NGCF could incorporate numerous redundant parameters, resulting in subpar results. Alternatively, BPRMF's straightforward and efficient design drives good performance. The SGL method, based on contrastive learning, outperforms the GNN-based methods NGCF and LightGCN, indicating the effectiveness of contrastive learning to achieve evenly distributed embeddings. Furthermore, the HGNN method, based on hypergraph neural networks, outperforms NGCF and LightGCN, demonstrating the potential of hypergraph neural networks in mitigating the over-smoothing issue when aggregating high-order neighbor information.

C. Layer Combination Analysis

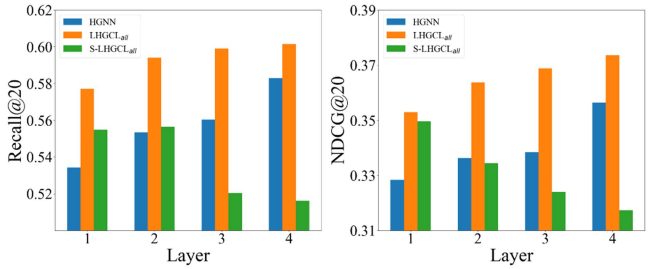
To verify the effectiveness of the proposed light hypergraph neural network framework, an investigation is conducted to identify the influence of diverse layer combination approaches on the performance of recommendations. Two variant components have been selected for further analysis:

1) *HGNN:* It is the first work that designs a hypergraph convolution operation in representation learning.
2) *LHGCL$_{all}$:* It is a light hypergraph neural network proposed in this paper, which removes transformation matrices and non-linear activations during message passing, and it removes all attribute information and the contrastive learning component.
3) *S-LHGCL$_{all}$(Single LHGCL$_{all}$):* It only uses the output of the last layer of LHGCL$_{all}$ and does not use layer combination.

Fig. 4 shows the optimal performance of Recall@20 and NDCG@20 at different layers of HGNN, LHGCL$_{all}$ and S-LHGCL$_{all}$, respectively. By concentrating on HGNN and LHGCL$_{all}$, it can be observed that their efficiency gradually improves with the addition of more layers. Their performance is not compromised, even when there are four layers. This confirms the potency of layer combination. LHGCL$_{all}$ consistently outperforms HGNN at each layer, indicating that transformation matrices and non-linear activations do not positively impact recommendation performance. When examining S-LHGCL$_{all}$, we observe a rapid decline in performance when layer $= 4$, highlighting the importance of neighbor information in each

[2][Online]. Available: https://pytorch.org/

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.



(a) Recall@20 (b) NDCG@20

Fig. 4. Impact of different layer combination methods, where the horizontal axis represents the layer size, and the vertical axis represents Recall@20 and NDCG@20, respectively.

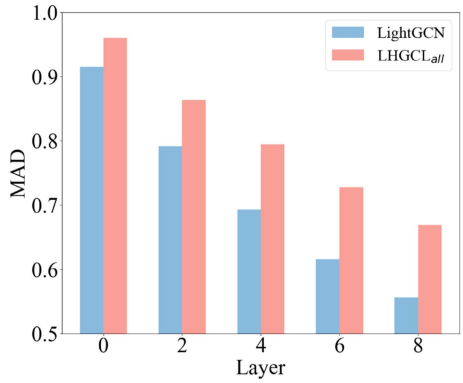
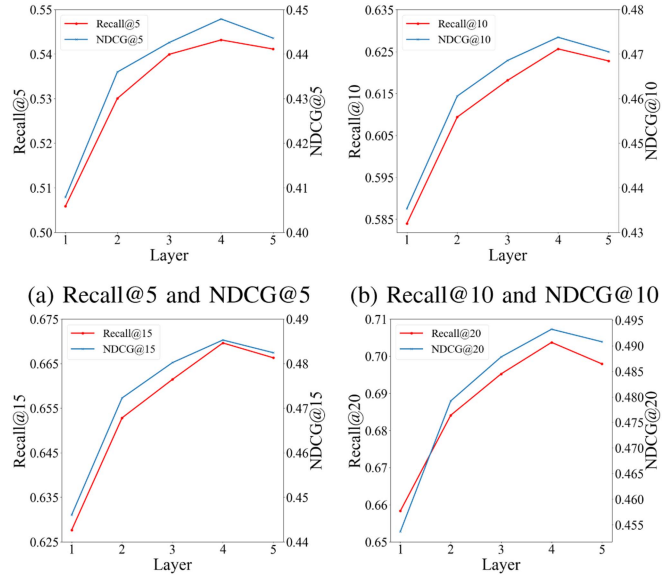


Fig. 5. MAD values of LHGCL$_{all}$ and LightGCN.

layer. Utilizing solely the information from the final layer leads to over-smoothing issues and poor performance.

D. Over-Smoothing Issue Analysis

To investigate whether hypergraph neural networks can effectively mitigate the over-smoothing issue, this section compares LHGCL$_{all}$ with LightGCN. LightGCN is a graph neural network that is the most similar model to LHGCL$_{all}$ since they both remove transformation matrices and non-linear activations during message passing. The degree of over-smoothing in the graph is quantified using the mean average distance (MAD) metric [48]. This metric reflects the smoothness of the graph representation by calculating the mean average distance among nodes. A higher MAD value indicates that the feature representations learned by the model are more distinguishable. The number of layers is set to 8 for both LHGCL$_{all}$ and LightGCN, and the MAD is calculated for the node representations of the even-numbered layers between 0 and 8. The 0th layer is the embedding layer, as illustrated in Fig. 5. It can be observed that as the graph layer increases, the MAD value of the node representations of LHGCL$_{all}$ consistently outperforms that of LightGCN at each layer. This indicates that LHGCL$_{all}$ mitigates the impact of the over-smoothing problem on the model performance. This is because hypergraph neural networks can effectively reduce the introduction of noise when aggregating high-order information.



(a) Recall@5 and NDCG@5 (b) Recall@10 and NDCG@10

(c) Recall@15 and NDCG@15 (d) Recall@20 and NDCG@20

Fig. 6. Impact of different layer size L, where the horizontal axis respresents the layer size (i.e., the numbers of propagation layers), and the vertical axes on the left and right represent Recall@K and NDCG@K, respectively. The values of K are 5, 10, 15, and 20.

E. Data Sparsity Issue Analysis

To verify the effectiveness of contrastive learning techniques in alleviating the data sparsity issue, we control the sparsity of the data by varying the size of the training set. Specifically, we randomly select from 50% to 80% of the data as the training set, while the remaining data is designated as the test set. We select the LightGCN, SGL, and LHGCL$_{all}$ methods and propose the variant LHGCL$_{cl}$, which incorporates the contrastive learning component based on the LHGCL$_{all}$ method. As evidenced by the experimental results presented in Table VII, it can be observed that SGL and LHGCL$_{cl}$ exhibit superior performance compared to LightGCN and LHGCL$_{all}$, respectively, across a range of training set sizes. The performance improvement is obvious when the training set is sparser. This indicates that LightGCN and LHGCL$_{all}$ may struggle to learn quality representations from sparse interaction data alone, and that the contrastive learning component has potential in addressing the data sparsity issue.

F. Hyperparameters Analysis

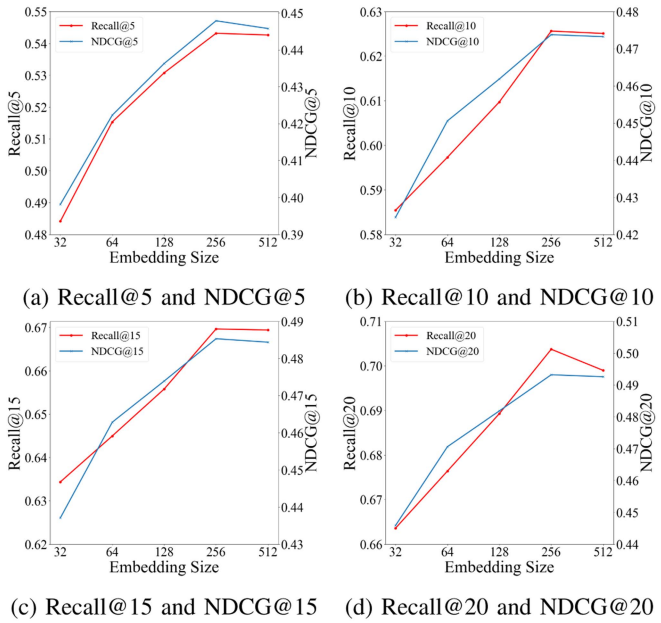
This section examines the impact of various pivotal hyperparameters (e.g., layer number L, embedding size d, regularization coefficient λ_1, the magnitude of contrastive learning λ_2) on the performance of our LHGCL approach. To isolate the effects of the hyperparameters, all other parameters are held constant and the hyperparameters are exclusively altered for the purposes of experimentation. The experimental results are presented in Figs. 6, 7, 8 and 9.

Impact of Layer Number: Incorporating different numbers of propagation layers enables an exploration of the LHGCL

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE COMPARISON OF DIFFERENT METHODS UNDER DIFFERENT SPARSITY TRAINING SETS

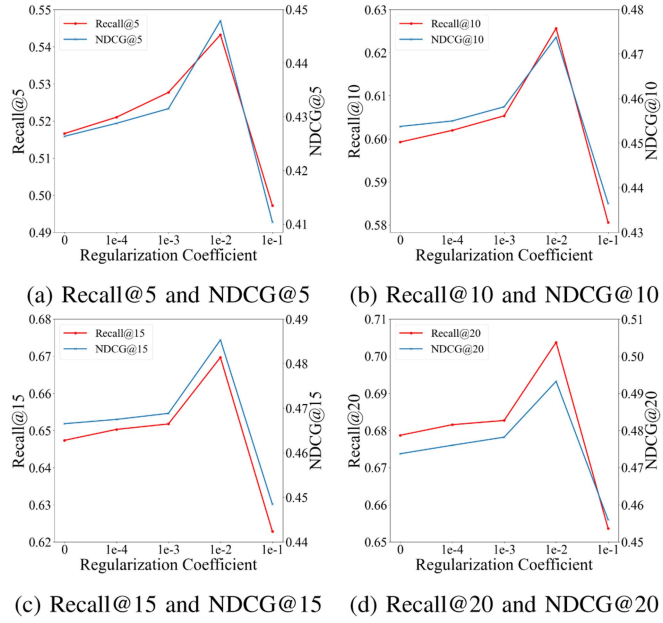
Methods	Different Sparsity Training Sets							
	50%		60%		70%		80%	
	Recall@20	NDCG@20	Recall@20	NDCG@20	Recall@20	NDCG@20	Recall@20	NDCG@20
LightGCN	0.46294	0.30219	0.46827	0.31408	0.54496	0.34802	0.56507	0.35493
SGL	0.48186	0.31086	0.47766	0.32308	0.54752	0.36131	0.56683	0.36435
LHGCL$_{all}$	0.48744	0.31355	0.49052	0.32814	0.55546	0.36943	0.60148	0.37360
LHGCL$_{cl}$	**0.52913**	**0.32897**	**0.53442**	**0.35223**	**0.57850**	**0.37605**	**0.60575**	**0.37602**
Improve.%	**8.6%**	**4.9%**	**8.9%**	**7.3%**	**4.1%**	**1.8%**	**0.7%**	**0.6%**



(a) Recall@5 and NDCG@5 (b) Recall@10 and NDCG@10

(c) Recall@15 and NDCG@15 (d) Recall@20 and NDCG@20

Fig. 7. Impact of different embedding size d, where the horizontal axis respresents the embedding size, and the vertical axes on the left and right represent Recall@K and NDCG@K, respectively. The values of K are 5, 10, 15, and 20.



(a) Recall@5 and NDCG@5 (b) Recall@10 and NDCG@10

(c) Recall@15 and NDCG@15 (d) Recall@20 and NDCG@20

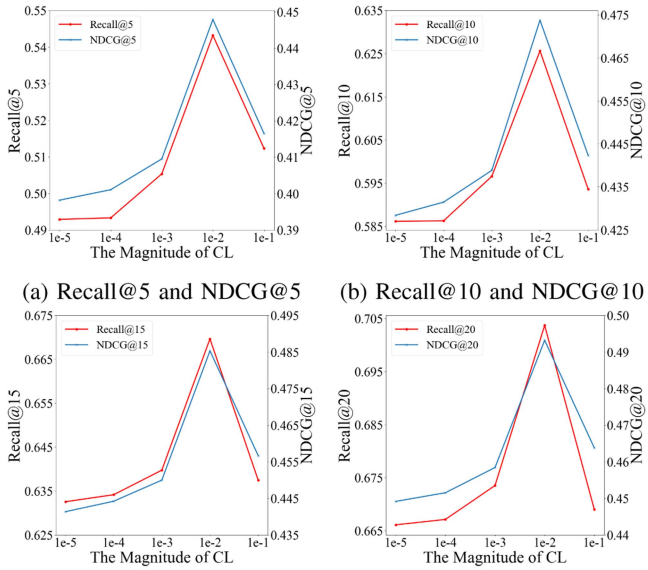
Fig. 8. Impact of different regularization coefficient λ_1, where the horizontal axis respresents the regularization coefficient, and the vertical axes on the left and right represent Recall@K and NDCG@K, respectively. The values of K are 5, 10, 15, and 20.

approach's potential to improve recommendation quality by propagating higher-order information. The experiments are conducted to test the approach with L values in the set $\{1, 2, 3, 4, 5\}$. The results are presented in Fig. 6. The findings indicate a clear correlation between the number of layers and the recommendation performance, with the optimal performance being achieved at $L = 4$. However, it is worth noting that the performance does not consistently improve as L increases to 5, suggesting that deeper models are more susceptible to encountering the issue of over-smoothing.

Impact of Embedding Size: The embedding size is of critical importance in determining the expression ability of the model. An embedding size that is too small may struggle to represent complex features, while one that is too large can result in noise and additional computational costs. The embedding size d is increased from 32 to 512, with the experimental results presented in Fig. 7. The experimental results indicate a noticeable enhancement in the recommendation performance as d increases from 32 to 256, particularly at $d = 256$, where LHGCL demonstrated the most exceptional recommendation performance. This is reasonable because we combine multiple attribute information of mashups and services to enable larger embedding sizes that better represent their complex features. However, when d is increased from 256 to 512, the performance of the proposed model decreases. This may be attributed to the excessive embedding size, which introduces noise during the representation learning process.

Impact of Regularization Coefficient: The purpose of the regularization term is to prevent the proposed model from overfitting. Our research investigates the impact of the regularization coefficient λ_1, which we vary within the set {0, 1e-4, 1e-3, 1e-2, 1e-1}. The results, depicted in Fig. 8, indicate that the proposed model performs better when λ_1 is gradually increased. This finding confirms that the regularization is effective in mitigating overfitting. We attain the best performance with λ_1 set at 1e-2. However, when λ_1 is set to 1e-1, the performance drops significantly, which indicates that too strong regularization will have negative effects on the model training.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.



(a) Recall@5 and NDCG@5 (b) Recall@10 and NDCG@10

(c) Recall@15 and NDCG@15 (d) Recall@20 and NDCG@20

Fig. 9. Impact of different magnitudes of contrastive learning λ_2, where the horizontal axis represents the magnitude of contrastive learning, and the vertical axes on the left and right represent Recall@K and NDCG@K, respectively. The values of K are 5, 10, 15, and 20.

Impact of the Magnitude of Contrastive Learning: To avoid the negative interference from the auxiliary task in gradient propagating, it is necessary to select relatively small values for λ_2. This is achieved by varying λ_2 within the set {0, 1e-4, 1e-3, 1e-2, 1e-1}. The results are presented in Fig. 9. It can be observed that as λ_2 increases, the recommendation performance of LHGCL improves, reaching a peak when $\lambda_2 =$ 1e-2. It can be concluded that even a very small λ_2 can promote the recommendation performance. However, when λ_2 is set to 1e-1, the performance declines significantly, indicating that the recommendation task is sensitive to the magnitude of contrastive learning. Therefore, choosing a small value is more likely to improve the recommendation performance.

G. Ablation Study

The objective of this study is to investigate the effect of the components in LHGCL and to validate whether each component contributes positively to the recommendation performance. To accomplish this, four variants of LHGCL are implemented:

- **LHGCL**$_{all}$: removes all attribute information and the contrastive learning component.
- **LHGCL**$_{c}$: introduces only the category attribute information and removes the contrastive learning component.
- **LHGCL**$_{cp}$: introduces the category and provider attribute information, and removes the contrastive learning component.
- **LHGCL**$_{attr}$: removes only the contrastive learning component of LHGCL.

The experimental results are shown in Tables VIII and IX. It can be observed that the introduction of any kind of attribute information leads to an improvement in recommendation performance. This indicates that attribute information has a positive effect on LHGCL, thereby improving the recommendation performance. Furthermore, the integration of the contrastive learning component has led to a notable improvement in recommendation performance, which substantiates the effectiveness of contrastive learning in learning meaningful and discriminative embeddings.

TABLE VIII
PERFORMANCE COMPARISON OF LHGCL AND ITS VARIANTS IN RECALL@K

Methods	Recall@5	Recall@10	Recall@15	Recall@20
LHGCL$_{all}$	0.42407	0.51936	0.56772	0.60148
LHGCL$_{c}$	0.47404	0.56258	0.61042	0.64170
LHGCL$_{cp}$	0.48071	0.56867	0.61518	0.64449
LHGCL$_{attr}$	0.49689	0.58752	0.63562	0.66532
LHGCL	**0.54319**	**0.62563**	**0.66965**	**0.70313**

TABLE IX
PERFORMANCE COMPARISON OF LHGCL AND ITS VARIANTS IN NDCG@K

Methods	NDCG@5	NDCG@10	NDCG@15	NDCG@20
LHGCL$_{all}$	0.32105	0.35238	0.36537	0.37360
LHGCL$_{c}$	0.37888	0.40889	0.42148	0.43013
LHGCL$_{cp}$	0.38175	0.40984	0.42292	0.43198
LHGCL$_{attr}$	0.40099	0.42983	0.44380	0.45159
LHGCL	**0.44580**	**0.47379**	**0.48531**	**0.49321**

VI. CONCLUSION

In this paper, we propose a light heterogeneous hypergraph contrastive learning based service recommendation method for mashup creation (LHGCL). Specifically, we first construct a heterogeneous hypergraph by combining the mashup information, service information, mashup-service interaction data, and their related attribute information. Next, we design a light hypergraph neural network to capture the high-order relationships between mashups and services, which mitigates the over-smoothing issue and leverages the attribute information. Finally, we use contrastive learning to improve the representations of mashups and services, which alleviates the issue of data sparsity and learns meaningful and discriminative feature representations for mashups and services. We conduct extensive experiments on a real-world dataset to evaluate the LHGCL method. Comprehensive experimental results demonstrate that the proposed method significantly outperforms other state-of-the-art methods. The design of a light hypergraph neural network, the introduction of attribute information, and the application of contrastive learning technology all contribute to the enhancement of the performance of the proposed model.

In future work, we plan to explore a broader range of heterogeneous features, such as mashup's tag information and service's tag information, to enrich our recommendation model and improve its performance. Furthermore, we will explore the potential of incorporating causal analysis into our methodology. This could help us uncover the underlying causal relationships between various features and mashup user preferences, offering a more subtle understanding of how various factors influence recommendations. By doing this, we expect to bolster our model's explainability and effectiveness, ultimately yielding more precise and personalized outcomes for mashup users.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] N. Yan, "Build your mashup with web services," in *Proc. IEEE 2007 Int. Conf. Web Serv.*, 2007, pp. xli–xli.

[2] Y. Jiang, J. Liu, M. Tang, and X. Liu, "An effective web service recommendation method based on personalized collaborative filtering," in *Proc. 2011 IEEE Int. Conf. Web Serv.*, 2011, pp. 211–218.

[3] H. Sun, Z. Zheng, J. Chen, W. Pan, C. Liu, and W. Ma, "Personalized open API recommendation in clouds via item-based collaborative filtering," in *Proc. IEEE 4th Int. Conf. Utility Cloud Comput.*, 2011, pp. 237–244.

[4] A. N. Ngaffo, W. El Ayeb, and Z. Choukair, "A service recommendation approach based on trusted user profiles and an enhanced similarity measure," *Electron. Commerce Res.*, vol. 22, pp. 1537–1572, 2022.

[5] P. Kirubanantham, S. U. Sankar, C. Amuthadevi, M. Baskar, M. S. Raja, and P. Karthik, "An intelligent web service group-based recommendation system for long-term composition," *J. Supercomputing*, vol. 78, no. 2, pp. 1944–1960, 2022.

[6] S. Barick, S. Das, and M. Parhi, "User-based cloud service recommendation system," in *Proc. 6th Int. Conf. Advance Comput. Intell. Eng.*, Springer, 2022, pp. 207–215.

[7] X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua, "Neural graph collaborative filtering," in *Proc. 42nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2019, pp. 165–174.

[8] L. Chen, L. Wu, R. Hong, K. Zhang, and M. Wang, "Revisiting graph based collaborative filtering: A linear residual graph convolutional network approach," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 27–34.

[9] W. Yu and Z. Qin, "Graph convolutional network for recommendation with low-pass collaborative filters," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10 936–10 945.

[10] B. Cao, L. Zhang, M. Peng, Y. Qing, G. Kang, and J. Liu, "Web service recommendation via combining bilinear graph representation and xDeepFM quality prediction," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1078–1092, Jun. 2023.

[11] M. Liu, Z. Tu, H. Xu, X. Xu, and Z. Wang, "DySR: A dynamic graph neural network based service bundle recommendation model for mashup creation," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2592–2605, Jul./Aug. 2023.

[12] Y. Jin, Y. Zhang, and Y. Zhang, "Neighbor library-aware graph neural network for third party library recommendation," *Tsinghua Sci. Technol.*, vol. 28, no. 4, pp. 769–785, 2023.

[13] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3558–3565.

[14] Y. Gao, Z. Zhang, H. Lin, X. Zhao, S. Du, and C. Zou, "Hypergraph learning: Methods and practices," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2548–2566, May 2022.

[15] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognit.*, vol. 110, 2021, Art. no. 107637.

[16] Z. Jia, Y. Fan, C. Wei, and R. Yan, "A spatial–temporal hypergraph based method for service recommendation in the mobile Internet of Things-enabled service platform," *Adv. Eng. Inform.*, vol. 57, 2023, Art. no. 102038.

[17] L. Zhang et al., "Construction and embedding representation of relational aggregation hypergraphs in cyber-physical systems," *IEEE Syst. J.*, vol. 17, no. 4, pp. 5216–5227, Dec. 2023.

[18] G. Xiao, J. Fei, D. Li, C. Wang, Z. Cheng, and J. Lu, "MRHN: Hypergraph convolutional network for web API recommendation," in *Proc. IEEE 24th Int. Conf. Inf. Reuse Integration Data Sci.*, 2023, pp. 179–184.

[19] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *Technologies*, vol. 9, no. 1, 2020, Art. no. 2.

[20] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.

[21] T. Yu, L. Zhang, H. Liu, H. Liu, and J. Wang, "Service recommendation based on contrastive learning and multi-task learning," *Comput. Commun.*, vol. 213, pp. 285–295, 2023.

[22] Y. Yang, C. Huang, L. Xia, and C. Li, "Knowledge graph contrastive learning for recommendation," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 1434–1443.

[23] C. Yang, J. Zou, J. Wu, H. Xu, and S. Fan, "Supervised contrastive learning for recommendation," *Knowl.-Based Syst.*, vol. 258, 2022, Art. no. 109973.

[24] J. Wu et al., "Self-supervised graph learning for recommendation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 726–735.

[25] X. Cai, C. Huang, L. Xia, and X. Ren, "LightGCL: Simple yet effective graph contrastive learning for recommendation," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–15.

[26] J. Shuai et al., "A review-aware graph contrastive learning framework for recommendation," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 1283–1293.

[27] Y. Zhang et al., "Enhancing sequential recommendation with graph contrastive learning," 2022, *arXiv:2205.14837*.

[28] M. Chen, C. Huang, L. Xia, W. Wei, Y. Xu, and R. Luo, "Heterogeneous graph contrastive learning for recommendation," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, 2023, pp. 544–552.

[29] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.

[30] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 974–983.

[31] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–14.

[32] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 639–648.

[33] X. Wang, H. Jin, A. Zhang, X. He, T. Xu, and T.-S. Chua, "Disentangled graph collaborative filtering," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 1001–1010.

[34] X. Zhang and M. Gan, "Hi-GNN: Hierarchical interactive graph neural networks for auxiliary information-enhanced recommendation," *Knowl. Inf. Syst.*, vol. 66, pp. 115–145, 2023.

[35] A. Bretto, "Hypergraph theory," in *An Introduction. Mathematical Engineering*, vol. 1. Cham, Switzerland: Springer, 2013.

[36] J. Wang, K. Ding, L. Hong, H. Liu, and J. Caverlee, "Next-item recommendation with sequential hypergraphs," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 1101–1110.

[37] S. Ji, Y. Feng, R. Ji, X. Zhao, W. Tang, and Y. Gao, "Dual channel hypergraph collaborative filtering," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 2020–2029.

[38] S. Peng, K. Sugiyama, and T. Mine, "Less is more: Reweighting important spectral graph features for recommendation," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 1273–1282.

[39] Z. Lin, C. Tian, Y. Hou, and W. X. Zhao, "Improving graph collaborative filtering with neighborhood-enriched contrastive learning," in *Proc. ACM Web Conf.*, 2022, pp. 2320–2329.

[40] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and Q. V. H. Nguyen, "Are graph augmentations necessary? Simple graph contrastive learning for recommendation," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 1294–1303.

[41] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.

[42] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertainty Artif. Intell.*, 2009, pp. 452–461.

[43] X. Xie et al., "Contrastive learning for sequential recommendation," in *Proc. IEEE 38th Int. Conf. Data Eng.*, 2022, pp. 1259–1273.

[44] M. Gutmann and A. Hyvärinen, "Noise-contrastive estimation: A new estimation principle for unnormalized statistical models," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 297–304.

[45] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–12.

[46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015, pp. 1–15.

[47] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.

[48] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 3438–3445.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.

Mingdong Tang (Member, IEEE) received the BS degree in electrical engineering from Tianjin University, Tianjin, China, in 2000, the MS degree in control engineering from Shanghai University, Shanghai, China, in 2003, and the PhD degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2010. He is currently a professor with the School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China. He has authored/coauthored more than 100 peer-reviewed scientific research papers in various journals and conferences. His research interests include service-oriented computing, data mining, and recommender systems.

Jiajin Mai received the bachelor's degree in software engineering from the Guangdong University of Foreign Studies, Guangzhou, China, in 2021. He is currently working toward the master's degree in computer science with the School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou, China. His research interests include recommender systems, services computing, software engineering, and deep learning.

Fenfang Xie received the PhD degree in computer science and technology from Sun Yat-sen University, Guangzhou, China, in 2019. She is currently a lecturer with the School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou. Her research interests include recommender systems, services computing, software engineering, machine learning, and deep neural network.

Zibin Zheng (Fellow, IEEE) received the PhD degree from the Chinese University of Hong Kong, Hong Kong, in 2011. He is currently a professor and the deputy dean with the School of Software Engineering, Sun Yat-sen University, Guangzhou, China. He authored or co-authored more than 200 international journal and conference papers, including one ESI hot paper and ten ESI highly cited papers. According to Google Scholar, his papers have more than 28000 citations. His research interests include blockchain, software engineering, and services computing. He was the recipient of several awards, including the Top 50 Influential Papers in Blockchain of 2018, the ACM SIGSOFT Distinguished Paper Award at ICSE2010, and the Best Student Paper Award at ICWS2010. He was the BlockSys'19 and CollaborateCom16 general co-chair, and the SC2'19, ICIOT18, and IoV14 PC co-chair. He is a fellow of the IET.

Authorized licensed use limited to: China Jiliang University. Downloaded on December 01,2025 at 09:29:14 UTC from IEEE Xplore. Restrictions apply.